

d) Round Robin (Experiment with different quantum sizes for RR algorithm)

```
#include <stdio.h>
#define MAX 100

int main() {
    int n, q_count;
    int at[MAX], bt[MAX], rt[MAX];
    int ct[MAX], tat[MAX], wt[MAX];
    int queue[MAX], visited[MAX];
    int front, rear, time, completed;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival Time and Burst Time:\n");
    for (int i = 0; i < n; i++) {
        printf("P%d AT BT: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }
    printf("\nEnter number of quantum values: ");
    scanf("%d", &q_count);
    int quantum[q_count];
    printf("Enter quantum values:\n");
    for (int i = 0; i < q_count; i++) {
        scanf("%d", &quantum[i]);
    }
    for (int q = 0; q < q_count; q++) {
        for (int i = 0; i < n; i++) {
            rt[i] = bt[i];
            visited[i] = 0;
        }
        front = rear = 0;
        time = 0;
        completed = 0;
        queue[rear++] = 0;
        visited[0] = 1;
        while (completed < n) {
            if (front == rear)
                break;
            int i = queue[front++];
            if (rt[i] > quantum[q]) {
                time += quantum[q];
                rt[i] -= quantum[q];
            }
            else {
                time += rt[i];
                rt[i] = 0;
                ct[i] = time;
                completed++;
            }
        }
    }
}
```

```

    }
    for (int j = 0; j < n; j++) {
        if (!visited[j] && at[j] <= time) {
            queue[rear++] = j;
            visited[j] = 1;
        }
    }
    if (rt[i] > 0)
        queue[rear++] = i;
    if (front == rear) {
        for (int j = 0; j < n; j++) {
            if (rt[j] > 0) {
                queue[rear++] = j;
                visited[j] = 1;
                time = at[j];
                break;
            }
        }
    }
}

float total_tat = 0, total_wt = 0;
printf("\nQuantum = %d\n", quantum[q]);
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i];
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
}
printf("Average TAT = %.2f\n", total_tat / n);
printf("Average WT = %.2f\n", total_wt / n);
}
return 0;
}

```

OUTPUT:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT BT: 1 8
P2 AT BT: 4 7
P3 AT BT: 2 3

Enter number of quantum values: 3
Enter quantum values:
2
5
3

Quantum = 2


| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 1  | 8  | 15 | 14  | 6  |
| P2  | 4  | 7  | 18 | 14  | 7  |
| P3  | 2  | 3  | 9  | 7   | 4  |


Average TAT = 11.67
Average WT = 5.67

Quantum = 5


| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 1  | 8  | 16 | 15  | 7  |
| P2  | 4  | 7  | 18 | 14  | 7  |
| P3  | 2  | 3  | 13 | 11  | 8  |


Average TAT = 13.33
Average WT = 7.33

Quantum = 3


| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 1  | 8  | 14 | 13  | 5  |
| P2  | 4  | 7  | 18 | 14  | 7  |
| P3  | 2  | 3  | 6  | 4   | 1  |


Average TAT = 10.33
Average WT = 4.33

Process returned 0 (0x0)   execution time : 39.092 s
Press any key to continue.
|
```